

Mimari

1. Mimari stil

Ana finansal uygulama ****modüler monolith**** olarak tutulur. Amaç Wallet, Transaction ve Ledger gibi aynı atomik para hareketine katılan bileşenleri erken microservice ayrıştırmasıyla distributed transaction problemine dönüştürmemektir.

Solution'ın ana projeleri:

```
src/
  FinWallet.Api
  FinWallet.Application
  FinWallet.Domain
  FinWallet.Infrastructure
  FinWallet.Shared.Contracts
  FinWallet.Shared.Web
  FinWallet.Gateway
simulators/
  FakeBank.Api
  FakeFraud.Api
  FakeCutoff.Api
  FakeCampaign.Api
  FakeCommunication.Api
tests/
  FinWallet.Application.Tests
```

2. Katman sorumlulukları

****Domain:**** Entity/value object/invariant/state transition ve finansal kurallar. HTTP, SQL veya provider DTO bilmez.

****Application:**** Use-case orchestration ve port/interface'ler. Domain'i kullanır; Infrastructure implementasyonlarına bağımlı değildir.

****Infrastructure:**** MSSQL, Redis, JWT token üretimi, provider HttpClient adapter'ları ve persistence implementasyonları.

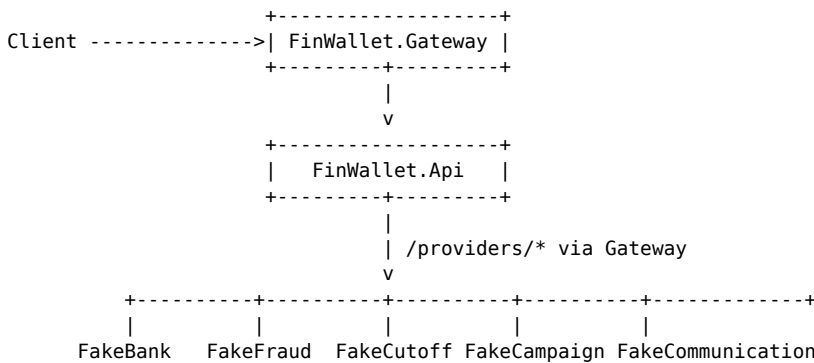
****Api:**** Controller, HTTP contract mapping, authentication/authorization composition ve exception mapping.

****Shared.Contracts:**** Ortak HTTP envelope ve simulatorlar arası paylaşılan stabil transport contractları.

****Shared.Web:**** Swagger, rate limit, Kestrel limits, CORS, security headers ve internal service-key kontrolü gibi ortak host davranışları.

****Gateway:**** YARP route/cluster, edge JWT, internal-service authorization, load balancing ve traffic controls.

3. Runtime topolojisi



Client normalde backend servislerini doğrudan çağırır. FinWallet.Api provider simulatorlarını da doğrudan çağırır; Gateway provider rotalarını kullanır.

4. Trust boundary'ler

- 1 ****Client -> Gateway:**** Public auth boundary. Protected /api/* rotalarında JWT gerekir.
- 2 ****Gateway -> FinWallet.Api:**** Gateway ayrı downstream service credential ekler. API ayrıca JWT ve ownership doğrular.
- 3 ****FinWallet.Api -> Gateway provider route:**** InternalServiceKey gerekir.
- 4 ****Gateway -> simulator:**** DownstreamServiceKey gerekir.

Bu yapı Gateway bypass edildiğinde business endpoint'in otomatik güvenilir hale gelmesini engeller.

5. Finansal transaction sınırı

Wallet-to-wallet transfer için aşağıdakiler tek MSSQL transaction içinde commit edilir:

- durable idempotency;
- source wallet debit;
- destination wallet credit;
- FinancialTransaction;
- LedgerJournal;
- LedgerEntries.

External HTTP bu SQL transaction açıkken çalıştırılmaz.

6. Bağımlılık yönü

Api -> Application -> Domain
Api -> Infrastructure -> Application/Domain
Gateway -> Shared.Web/Shared.Contracts
Simulators -> Shared.Web/Shared.Contracts

Domain'in Infrastructure/API'ye dependency alması yasaktır. Application provider-specific DTO veya SQL implementation bilmez.

7. Neden microservice değil?

Bugünkü sınırlar deployment bağımsızlığı için değil, doğruluk ve anlaşılabilirlik için seçilmiştir. Domain modülleri yeterince stabil olduğunda ve bağımsız scale/team ownership ihtiyacı olduğunda servis ayrıştırması değerlendirilebilir. Wallet/Ledger gibi aynı atomik commit'e ihtiyaç duyan bölümlerin ayrılması özellikle yüksek maliyetlidir.